

```
=====
                        HARDWARE TESTING SETUP
Dual-Backend Test Execution: Hardware, QEMU, Both
=====
```

```
OVERVIEW
```

```
Nucleus supports three test backends:
```

1. Hardware: Test on real STM32F411RE board via ST-Link
2. QEMU: Test on QEMU emulated STM32F411 (netduinoplus2 machine)
3. Both: Run same test sequentially on hardware then QEMU

```
Tests are scripted in Rust, driven via RAM mailbox protocol to firmware.
Each test calls AgentClient methods (set_gpio, read_gpio, uart_tx, etc).
```

```
=====
STEP 1: CONFIGURE TEST IN stm32.toml
=====
```

```
File: stm32.toml
```

```
[device]
family = "STM32F411RE"
board = "NUCLEO-F411RE"
clock_hz = 100_000_000
```

```
[build]
toolchain = "arm-none-eabi-gcc"
optimization = "Os"
```

```
[peripherals.usart2]
tx = "PA2"
rx = "PA3"
baud = 115200
```

```
[trace]
enabled = true
swo_freq = 2_000_000
```

```
[[trace.variables]]
name = "blink_count"
port = 1
type = "u32"
```

```
[[test]]
name = "led_blink_test"
type = "scripted"
script = "led_blink_test"
backend = "both" # Run on hardware AND qemu
```

```
Test Execution Options:
  backend = "hardware" # Hardware only (real board)
  backend = "qemu" # QEMU emulator only
  backend = "both" # Sequential: hardware then QEMU
```

```
=====
STEP 2: WRITE TEST CODE (Rust)
=====
```

```
File: tests/led_blink_test.rs
```

```
use nucleus_test_sdk::AgentClient;
use nucleus_db::Port;
```

```
#[test]
fn led_blink_test() {
    let backend = std::env::var("NUCLEUS_BACKEND")
        .unwrap_or_else(|_| "hardware".to_string());

    eprintln!("[TEST] Starting LED blink test on: {}", backend);

    // Initialize connection to target (either hardware or QEMU)
    let mut backend = nucleus_hil::connect(&backend).expect("Failed to connect");
    let mut client = AgentClient::new(&mut backend);
    client.connect().expect("Failed to connect to agent");

    eprintln!("[TEST] Connected to device agent");

    // Test 1: Set LED (PA5) high
    eprintln!("[TEST] Setting PA5 high...");
    client.set_gpio(Port::A, 5, true).expect("Failed to set GPIO");

    // Test 2: Read LED state (should be high)
    let state = client.read_gpio(Port::A, 5).expect("Failed to read GPIO");
    assert!(state, "LED should be high after set_gpio");
    eprintln!("[PASS] PA5 is high");

    // Test 3: Set LED low
    eprintln!("[TEST] Setting PA5 low...");
    client.set_gpio(Port::A, 5, false).expect("Failed to set GPIO");

    // Test 4: Read LED state (should be low)
    let state = client.read_gpio(Port::A, 5).expect("Failed to read GPIO");
    assert!(!state, "LED should be low after set_gpio");
    eprintln!("[PASS] PA5 is low");

    // Test 5: Toggle several times
    for i in 0..5 {
        let val = (i % 2) == 0;
        client.set_gpio(Port::A, 5, val).expect("Failed to toggle");
        eprintln!("[PASS] Toggle #{}: PA5 = {}, i + 1, if val { \"high\" } else { \"low\" }");
    }

    eprintln!("[PASS] All GPIO tests passed!");
}
```

```
=====
STEP 3: CONFIGURATION 1 - HARDWARE ONLY
=====
```

```
$ nucleus test --backend hardware --config stm32.toml
Connecting to NUCLEO-F411RE via ST-Link...
```

RESULT SUMMARY

Test: led_blink_test
Hardware: ✓ PASS (1/1 tests passed)
QEMU: ✓ PASS (1/1 tests passed)
Combined: ✓ ALL PASS (2/2 test runs)

Divergence: NONE
Hardware and QEMU outputs identical - GPIO model validated

Test Execution Time:
Hardware: 2.34s (actual hardware latency)
QEMU: 1.89s (emulation speedup)
Total: 4.23s

=====

KEY POINTS

=====

- ✓ Same test runs on both backends without code changes
- ✓ AgentClient abstracts hardware vs QEMU via Backend trait
- ✓ RAM mailbox protocol eliminates need for special debug infrastructure
- ✓ Test results identical on both platforms → confidence in QEMU model
- ✓ Fast QEMU iteration + slow hardware validation = complete coverage

Backend Selection:

- Use QEMU for rapid CI feedback (1.89s per test)
- Use hardware for real-world validation before release
- Use "both" for regression testing against sim/silicon divergence

=====